

UNIT-1

Operating System Overview

OVER VIEW OF OPERATING SYSTEM

What is an Operating System?

A program that acts as an intermediary between a user of a computer and the computer hardware

Operating system goals:

- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use the computer hardware in an efficient manner

Computer System Structure

- Computer system can be divided into four components
- Hardware – provides basic computing resources
- CPU, memory, I/O devices

Operating system

Controls and coordinates use of hardware among various applications and users

Application programs – define the ways in which the system resources are used to solve the computing problems of the users Word processors, compilers, web browsers, database systems, video games

Users

People, machines, other computers

Four Components of a Computer System

Operating System Definition

- OS is a resource allocator
- Manages all resources
- Decides between conflicting requests for efficient and fair resource use
- OS is a control program
- Controls execution of programs to prevent errors and improper use of the computer
- No universally accepted definition
- Everything a vendor ships when you order an operating system” is good approximation

But varies wildly.

- “The one program running at all times on the computer” is the kernel. Everything else is either a system program (ships with the operating system) or an application program

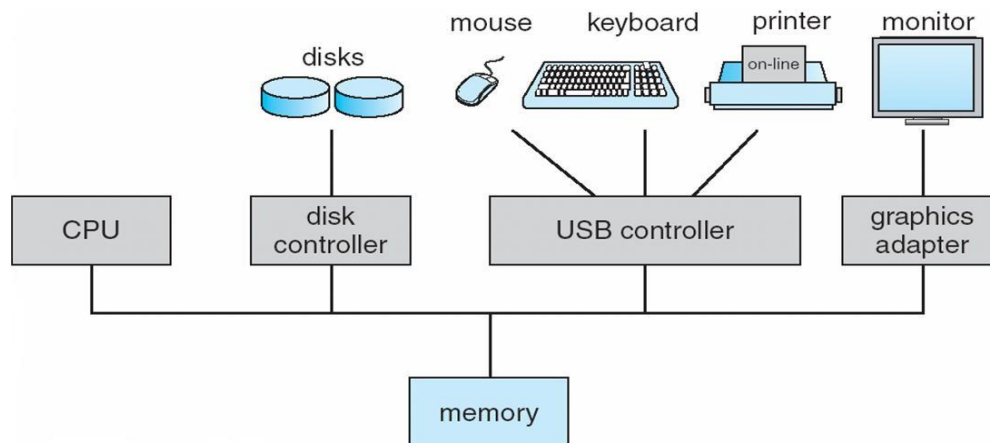
Computer Startup

- bootstrap program is loaded at power-up or reboot
- Typically stored in ROM or EPROM, generally known as firmware
- Initializes all aspects of system
- Loads operating system kernel and starts execution

Computer System Organization

- Computer-system operation

- One or more CPUs, device controllers connect through common bus providing access to shared memory
- Concurrent execution of CPUs and devices competing for memory cycles



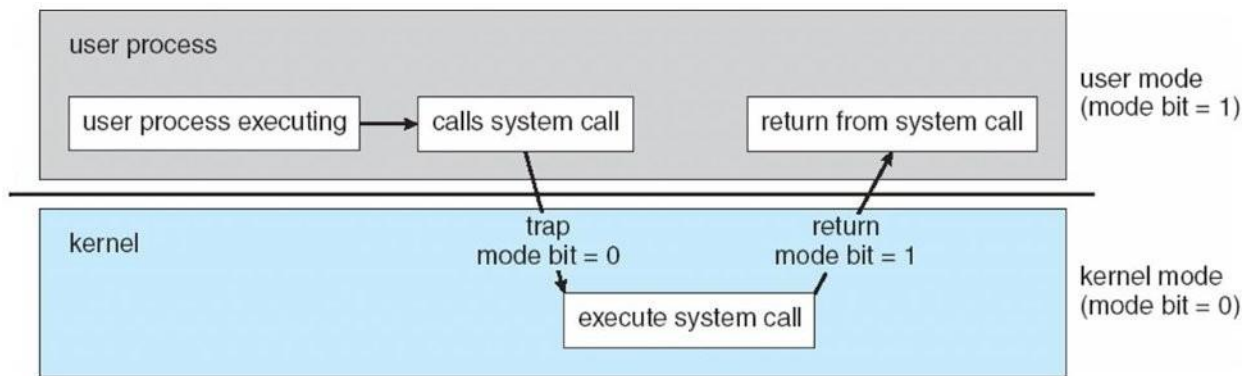
Operating-System Operations

- Interrupt driven by hardware
- Software error or request creates **exception** or **trap**
- Division by zero, request for operating system service
- Other process problems include infinite loop, processes modifying each other or the operating system
- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
 - **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code
 - Some instructions designated as **privileged**, only executable in kernel mode
 - System call changes mode to kernel, return from call resets it to user

Transition from User to Kernel Mode

- ✓ Timer to prevent infinite loop / process hogging resources
 - Set interrupt after specific period
 - Operating system decrements counter
 - When counter zero generate an interrupt

Set up before scheduling process to regain control or terminate program that exceeds allotted



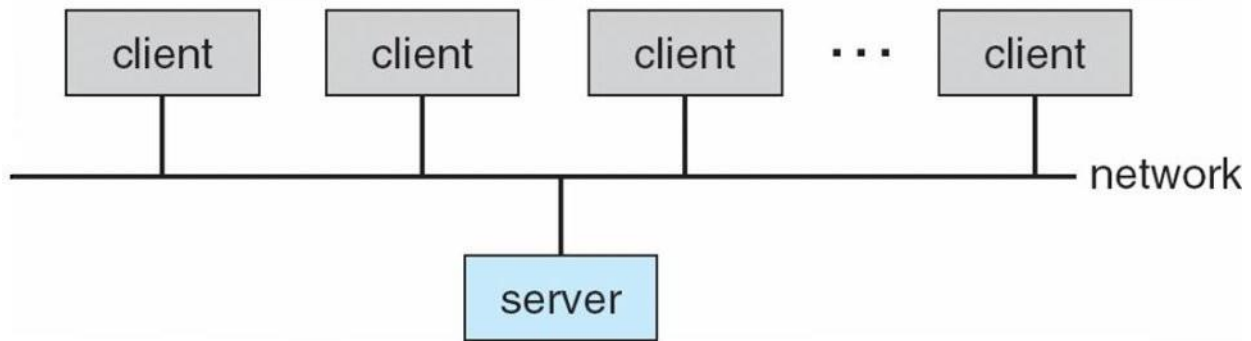
Protection and Security:

- ✓ **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- ✓ **Security** – defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
 - Systems generally first distinguish among users, to determine who can do what
 - User identities (**user IDs**, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation** allows user to change to effective ID with more rights

Computing Environments:

Client-Server Computing

- Dumb terminals supplanted by smart PCs
- Many systems now **servers**, responding to requests generated by **clients**
 - **Compute-server** provides an interface to client to request services (i.e., database)
 - **File-server** provides interface for clients to store and retrieve files



Peer to Peer:

- ✓ P2P does not distinguish clients and servers
 - Instead all nodes are considered peers
 - May each act as client, server or both
 - Node must join P2P network
 - Registers its service with central lookup service on network, or
 - Broadcast request for service and respond to requests for service via **discovery protocol**
 - Examples include *Napster* and *Gnutella*

Web-Based Computing

- ✓ Web has become ubiquitous
- ✓ PCs most prevalent devices
- ✓ More devices becoming networked to allow web access
- ✓ New category of devices to manage web traffic among similar servers: **load balancers**
- ✓ Use of operating systems like Windows 95, client-side, have evolved into Linux and Windows XP, which can be clients and servers

Open-Source Operating Systems:

- ✓ Operating systems made available in source-code format rather than just binary **closed-source**
- ✓ Counter to the **copy protection** and **Digital Rights Management (DRM)** movement
- ✓ Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**

- ✓ Examples include **GNU/Linux** and **BSD UNIX**(including core of **Mac OS X**), and many more

Operating System Services:

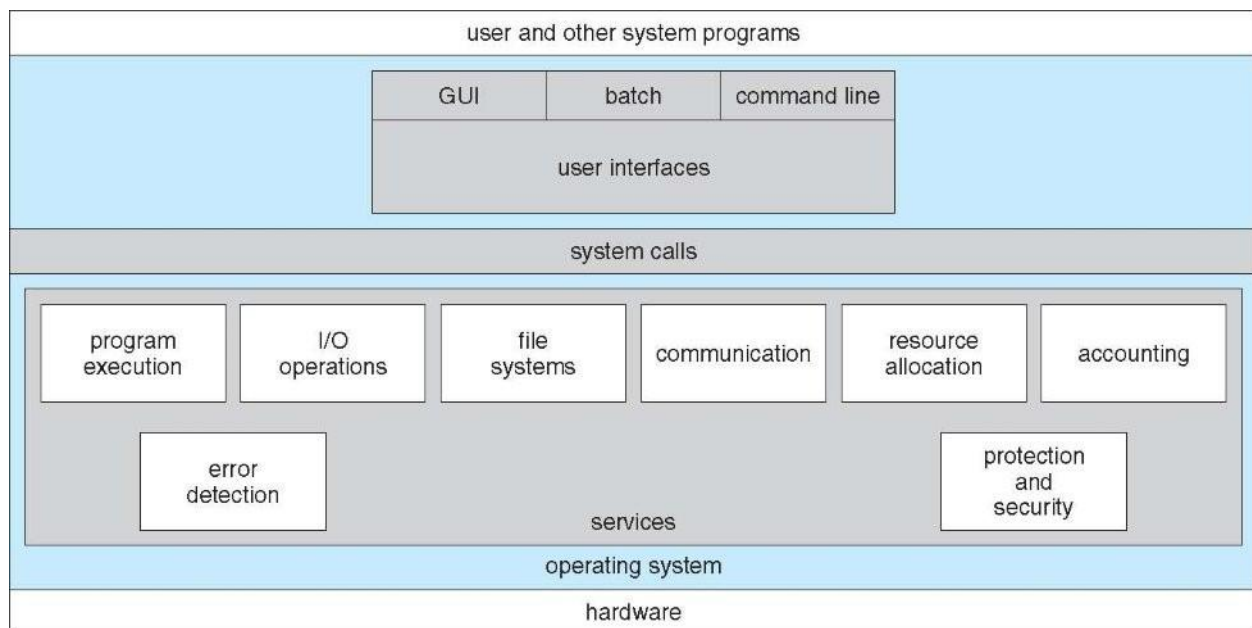
- ✓ Operating systems provide an environment for execution of programs and services to programs and users
- ✓ One set of operating-system services provides functions that are helpful to the user:
 - **User interface** - Almost all operating systems have a user interface(UI).
 - 4 Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
 - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
 - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file information, permission management.
 - **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - 4 Communications may be via shared memory or through message passing (packets moved by the OS)
 - **Error detection** – OS needs to be constantly aware of possible errors
 - 4 May occur in the CPU and memory hardware, in I/O devices, in user program
 - 4 For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - 4 Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system
- ✓ Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
 - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - 4 Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code

- **Accounting** - To keep track of which users use how much and what kinds of computer resources
- **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other

4 **Protection** involves ensuring that all access to system resources is controlled

4 **Security** of the system from outsiders requires user authentication, extend to defending external I/O devices from invalid access attempts

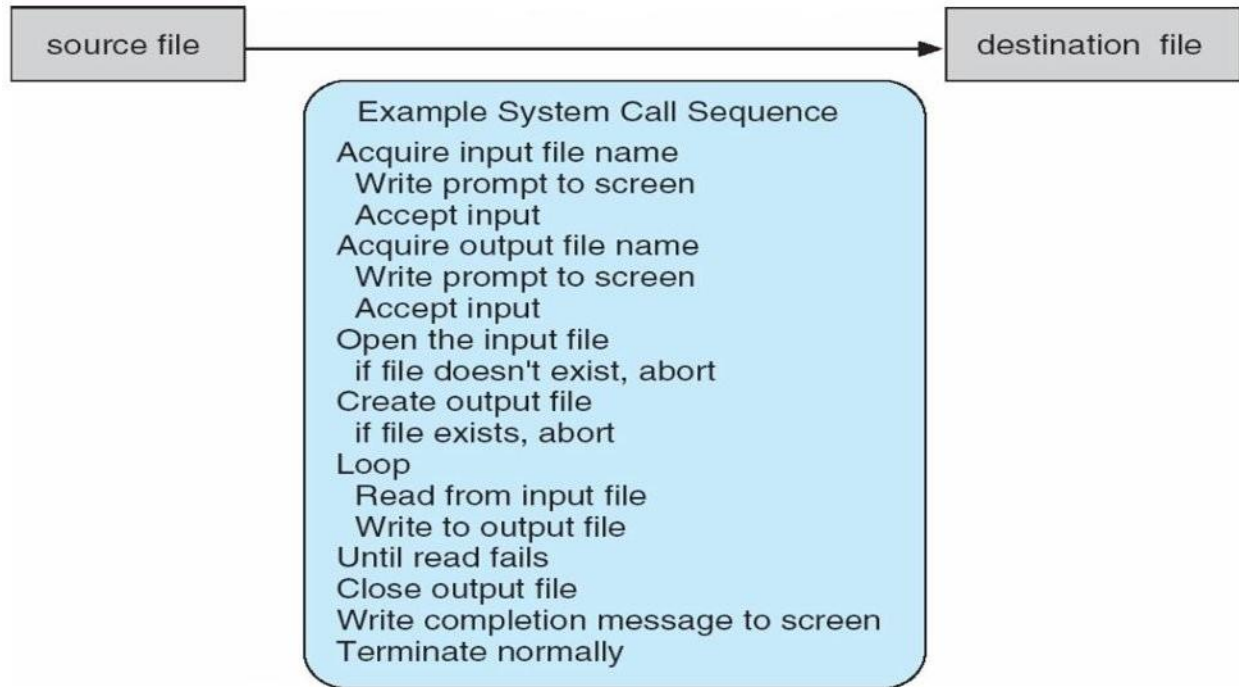
4 If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.



System Calls:

- ✓ Programming interface to the services provided by the OS
- ✓ Typically written in a high-level language (C or C++)
- ✓ Mostly accessed by programs via a high-level **Application Program Interface (API)** rather than direct system call use
- ✓ Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)
- ✓ Why use APIs rather than system calls?

(Note that the system-call names used throughout this text are generic)



Types of System Calls:

- ✓ Process control
 - end, abort
 - load, execute
 - create process, terminate process
 - get process attributes, set processattributes
 - wait for time
 - wait event, signal event
 - allocate and free memory
- ✓ File management
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- ✓ Device management
 - request device, release device

- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices
- ✓ Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- ✓ Communications
 - create, delete communication connection
 - send, receive messages
 - transfer status information
 - attach and detach remote devices

System Programs:

- ✓ System programs provide a convenient environment for program development and execution. They can be divided into:
 - File manipulation
 - Status information
 - File modification
 - Programming language support
 - Program loading and execution
 - Communications
 - Application programs
 - Most users' view of the operation system is defined by system programs, not the actual system calls
- ✓ Provide a convenient environment for program development and execution
 - Some of them are simply user interfaces to system calls; others are considerably more complex
 - **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- ✓ **Status information**
 - Some ask the system for info - date, time, amount of available memory, disk space, number of users
 - Others provide detailed performance, logging, and debugging information

- Typically, these programs format and print the output to the terminal or other output devices
- Some systems implement a registry - used to store and retrieve configuration information
- ✓ **File modification**
 - Text editors to create and modify files
 - Special commands to search contents of files or perform transformations of the text
 - **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- ✓ **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- ✓ **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

Operating-System Debugging:

- ✓ **Debugging** is finding and fixing errors, or **bugs**
- ✓ OSes generate **log files** containing error information
- ✓ Failure of an application can generate **core dump** file capturing memory of the process
- ✓ Operating system failure can generate **crash dump** file containing kernel memory
- ✓ Beyond crashes, performance tuning can optimize system performance
- ✓ Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."
- ✓ DTrace tool in Solaris, FreeBSD, Mac OS X allows live instrumentation on production systems
 - **Probes** fire when code is executed, capturing state data and sending it to consumers of those probes

Operating System Generation:

- ✓ Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site
- ✓ SYSGEN program obtains information concerning the specific configuration of the hardware system

- ✓ *Booting* – starting a computer by loading the kernel
- ✓ *Bootstrap program* – code stored in ROM that is able to locate the kernel, load it into memory, and start its execution

System Boot

- ✓ Operating system must be made available to hardware so hardware can start it
 - Small piece of code – **bootstrap loader**, locates the kernel, loads it into memory, and starts it
 - Sometimes two-step process where **boot block** at fixed location loads bootstrap loader
 - When power initialized on system, execution starts at a fixed memory location
- Firmware used to hold initial boot code

UNIT-2

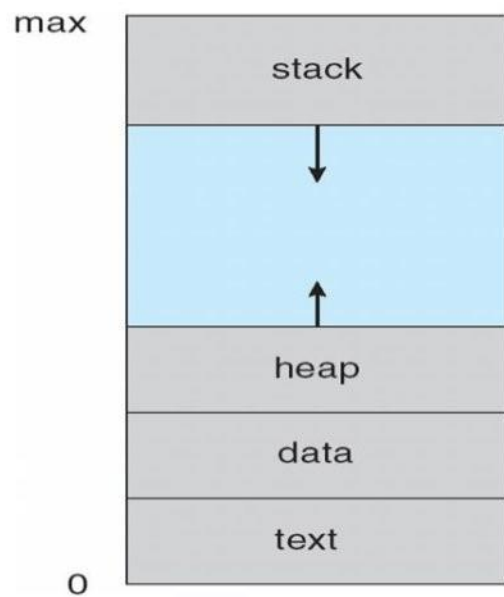
PROCESS THREADS, PROCESS SYNCHRONISATION, CPU SCHEDULING

Process Concept:

- ✓ An operating system executes a variety of programs:
 - Batch system – jobs
 - Time-shared systems – user programs or tasks
 - Textbook uses the terms *job* and *process* almost interchangeably
- ✓ Process – a program in execution; process execution must progress in sequential fashion
- ✓ A process includes:
 - program counter
 - stack
 - data section

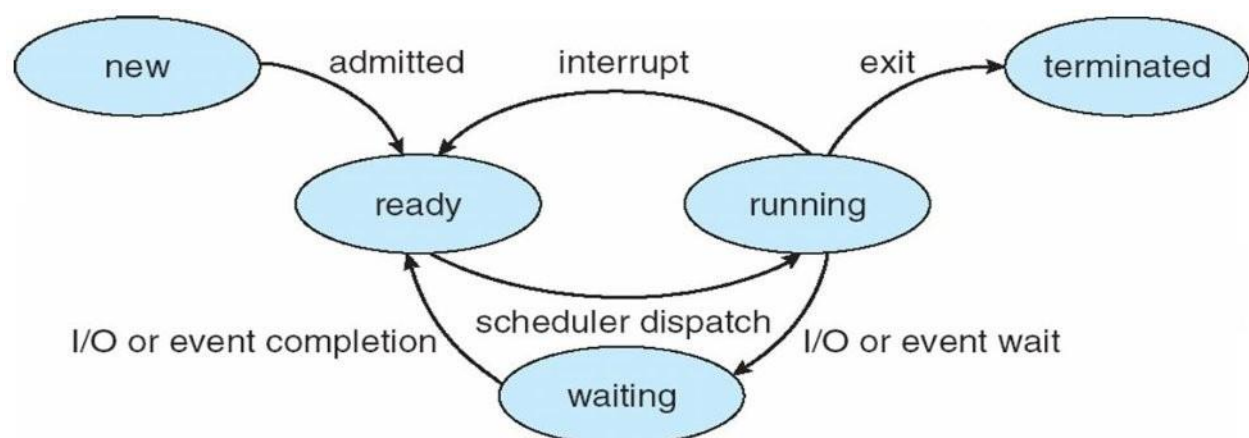
The Process:

- ✓ Multiple parts
 - The program code, also called **text section**
 - Current activity including **program counter**, processor registers
 - **Stack** containing temporary data
 - Function parameters, return addresses, local variables
 - **Data section** containing global variables
 - **Heap** containing memory dynamically allocated during runtime
- ✓ Program is passive entity, process is active
 - Program becomes process when executable file loaded into memory
- ✓ Execution of program started via GUI mouse clicks, command line entry of its name, etc
- ✓ One program can be several processes
 - Consider multiple users executing the same program



Process State:

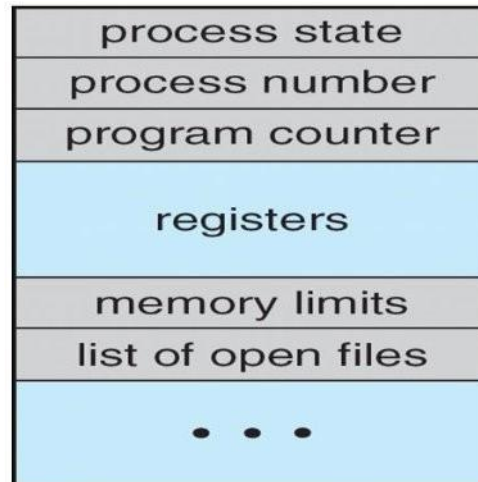
- ✓ As a process executes, it changes *state*
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution



Process Control Block (PCB):

Information associated with each process

- ✓ Process state
- ✓ Program counter
- ✓ CPU registers
- ✓ CPU scheduling information
- ✓ Memory-management information
- ✓ Accounting information
- ✓ I/O status information

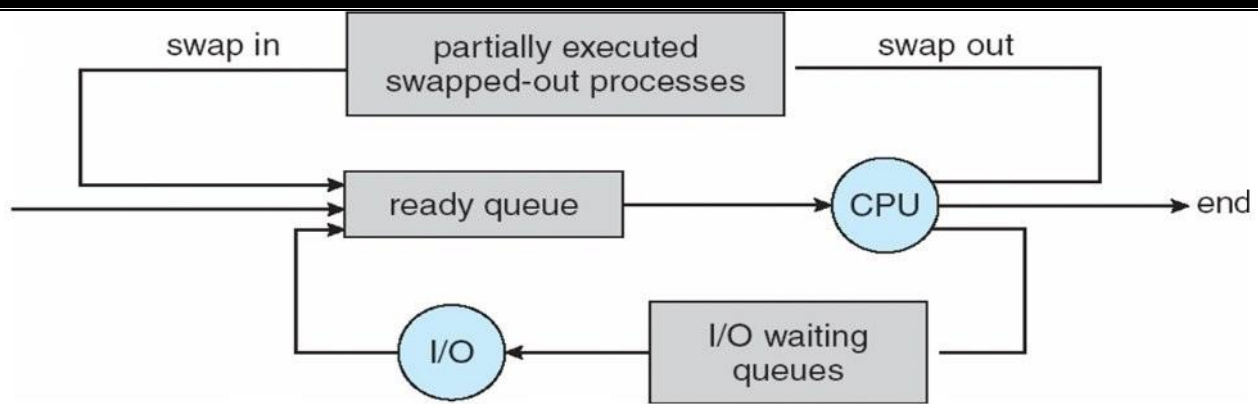


Process Scheduling:

- ✓ Maximize CPU use, quickly switch processes onto CPU for timesharing
- ✓ Process scheduler selects among available processes for next execution on CPU
- ✓ Maintains scheduling queues of processes

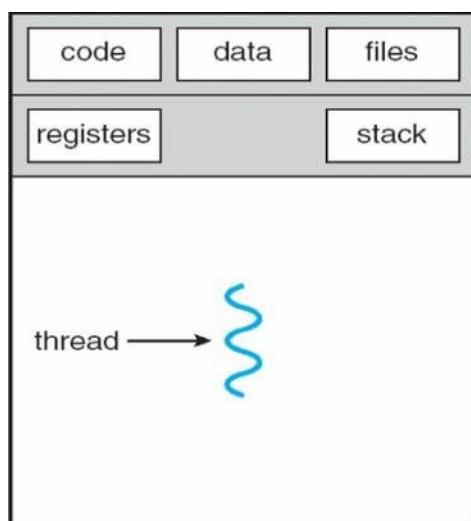
- Job queue – set of all processes in the system
- Ready queue – set of all processes residing in main memory, ready and waiting to execute
- Device queues – set of processes waiting for an I/O device
- Processes migrate among the various queues

Schedulers:

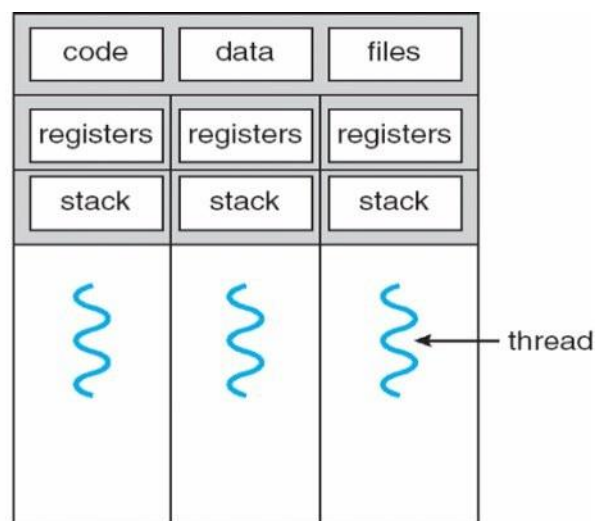


Threads

- ✓ Threads run within application
- ✓ Multiple tasks with the application can be implemented by separate threads
 - Update display
 - Fetch data
 - Spell checking
 - Answer a network request
- ✓ Process creation is heavy-weight while thread creation is light-weight
- ✓ Can simplify code, increase efficiency
- ✓ Kernels are generally multithreaded

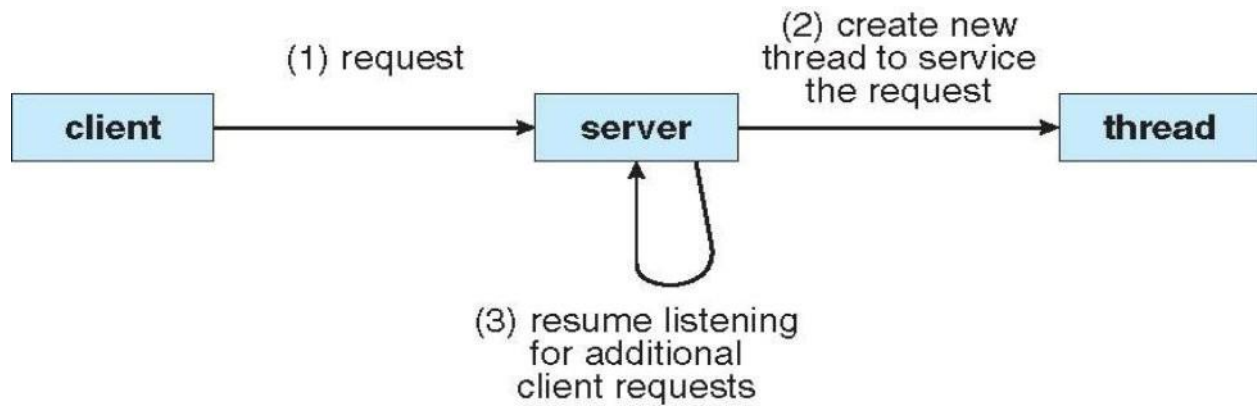


single-threaded process

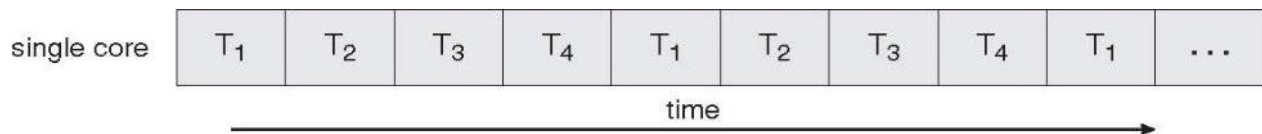


multithreaded process

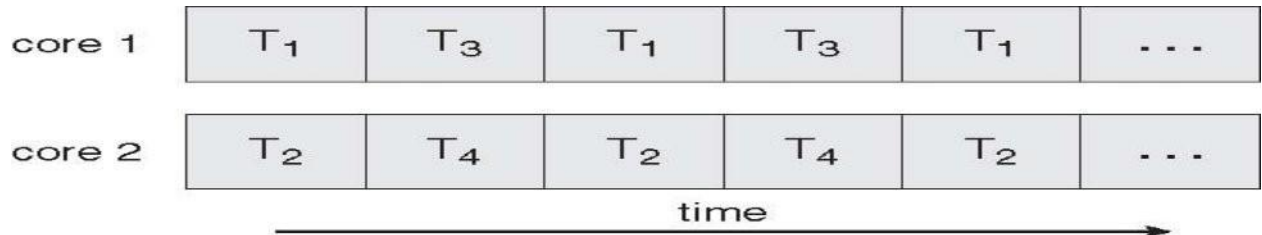
Multithreaded Server Architecture:



Concurrent Execution on a Single-core System



Parallel Execution on a Multicore System



User Threads:

- Σ Thread management done by user-level threads library
- Σ Three primary thread libraries:
 - POSIX **Pthreads**
 - Win32 threads
 - Java threads

Kernel Threads:

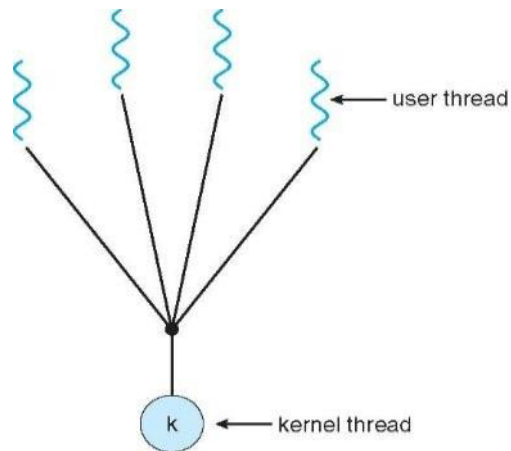
- Σ Supported by the Kernel
- Σ Examples
 - Windows XP/2000
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X

Multithreading Models:

- Σ Many-to-One
- Σ One-to-One
- Σ Many-to-Many

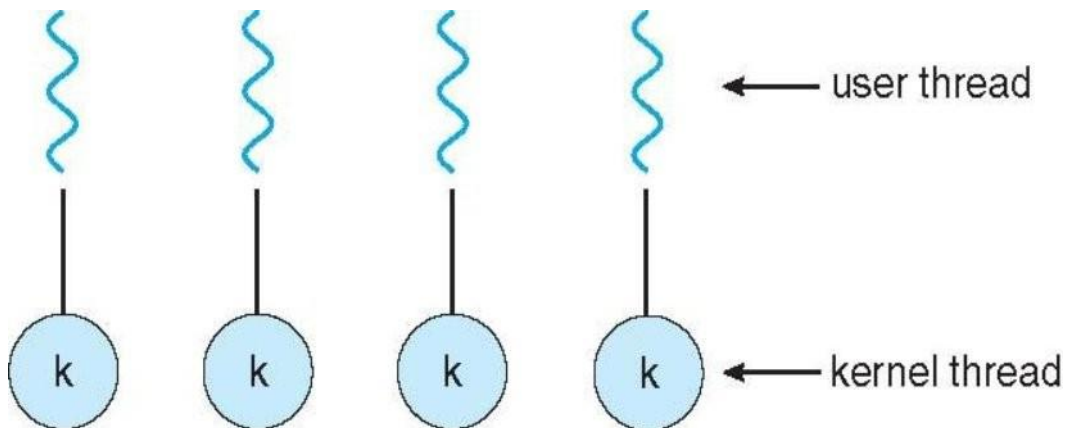
Many-to-One

- Σ Many user-level threads mapped to single kernel thread
- Σ Examples:
 - Solaris Green Threads
 - GNU Portable Threads



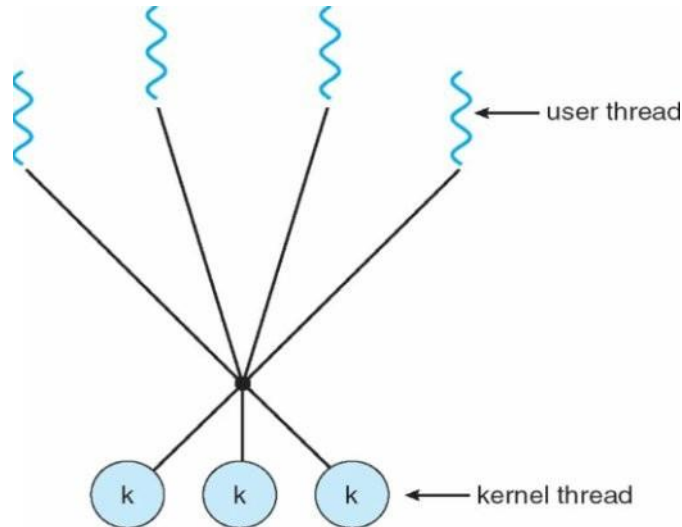
One-to-One:

- Σ Each user-level thread maps to kernel thread
- Σ Examples
 - Windows NT/XP/2000
 - Linux
 - Solaris 9 and later



Many-to-Many Model:

- Σ Allows many user level threads to be mapped to many kernel threads
- Σ Allows the operating system to create a sufficient number of kernel threads
- Σ Solaris prior to version 9
- Σ Windows NT/2000 with the *ThreadFiber* package



Thread Libraries:

- Σ **Thread library** provides programmer with API for creating and managing threads
- Σ Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS

Pthreads

- Σ May be provided either as user-level or kernel-level
- Σ A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- Σ API specifies behavior of the thread library, implementation is up to development of the library
- Σ Common in UNIX operating systems (Solaris, Linux, Mac OSX)

```

#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}

```

Java Threads:

- Σ Java threads are managed by the JVM
- Σ Typically implemented using the threads model provided by underlying OS
- Σ Java threads may be created by:
 - Extending Thread class
 - Implementing the Runnable interface

```

class Sum
{
    private int sum;

    public int getSum() {
        return sum;
    }

    public void setSum(int sum) {
        this.sum = sum;
    }
}

class Summation implements Runnable
{
    private int upper;
    private Sum sumValue;

    public Summation(int upper, Sum sumValue) {
        this.upper = upper;
        this.sumValue = sumValue;
    }

    public void run() {
        int sum = 0;
        for (int i = 0; i <= upper; i++)
            sum += i;
        sumValue.setSum(sum);
    }
}

```

Threading Issues:

- Σ Semantics of **fork()** and **exec()** systemcalls

Σ **Thread cancellation of targetthread**

- Asynchronous or deferred
- **Signalhandling**
- Synchronous and asynchronous

Σ **Thread pools**

Σ **Thread-specific data**

- n Create Facility needed for data private to thread

Σ **Scheduler activations**

Thread Cancellation:

- Σ Terminating a thread before it has finished
- Σ Two general approaches:
 - Asynchronous cancellation terminates the target thread immediately.
 - Deferred cancellation allows the target thread to periodically check if it should be cancelled.

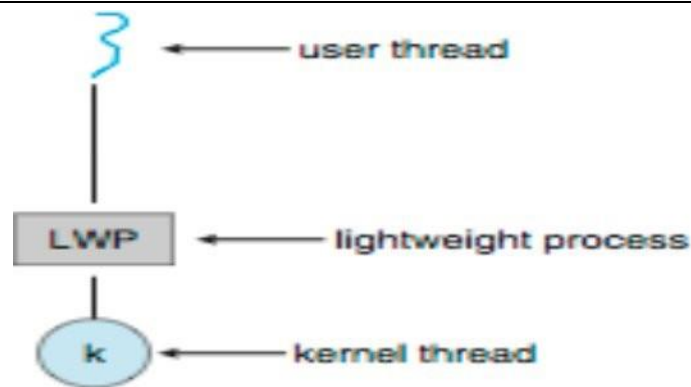
Thread Pools:

- Σ Create a number of threads in a pool where they await work
- Σ Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool

Scheduler Activations:

- Σ Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Σ Scheduler activations provide upcalls - a communication mechanism from the kernel to the thread library
- Σ This communication allows an application to maintain the correct number kernelthreads

Lightweight Processes



Critical Section Problem:

- Σ Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Σ Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process is in critical section, no other may be in its critical section
- Σ Critical section problem is to design protocol to solve this
- Σ Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- Σ Especially challenging with preemptive kernels

General structure of process P_i is

```

do {
    entry section
    critical section
    exit section
    remainder section
} while (TRUE);
  
```

Figure 6.1 General structure of a typical process P_i .

Solution to Critical-Section Problem:

1. Mutual Exclusion - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections

2. Progress - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely

3. Bounded Waiting - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

- ó Assume that each process executes at a nonzerospeed

- ó No assumption concerning relative speed of the nprocesses

Peterson's Solution:

- Σ Two process solution

- Σ Assume that the LOAD and STORE instructions are atomic; that is, cannot beinterrupted

- Σ The two processes share two variables:

- int**turn**;

- Boolean**flag[2]**

- The variable **turn** indicates whose turn it is to enter the criticalsection

- Σ The **flag** array is used to indicate if a process is ready to enter the critical section.**flag[i]** = true implies that process **P_i** is ready!

Algorithm for Process P_i

do {

 flag[i] = TRUE;

 turn = j;

 while (flag[j] && turn == j);

 critical section

 flag[i] = FALSE;

 remainder section

 } while (TRUE);

- Σ Provable that

- 1. Mutual exclusion is preserved

- 2. Progress requirement is satisfied

- 3. Bounded-waiting requirement is met

Synchronization Hardware:

- Σ Many systems provide hardware support for critical sectioncode

Σ Uniprocessors – could disable interrupts

- Currently running code would execute without preemption
- Generally too inefficient on multiprocessor systems

4 Operating systems using this not broadly scalable

4 Modern machines provide special atomic hardware instructions

4 Atomic = non-interruptable

- Either test memory word and set value
- Or swap contents of two memory words

do {

 acquire lock

 critical section

 release lock

 remainder section

} while (TRUE);

Semaphore:

Σ Synchronization tool that does not require busy waiting

Σ Semaphore S – integer variable

Σ Two standard operations modify S: wait() and signal()

- Originally called P() and V()

Σ Less complicated

Σ Can only be accessed via two indivisible (atomic) operations

wait (S) {

 while S <= 0 ; // no-op

 S--; }

signal (S) {

 S++;

}

Semaphore as General Synchronization Tool

Σ **Counting** semaphore – integer value can range over an unrestricted domain

- Σ **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement

1 Also known as **mutexlocks**

- Σ Can implement a counting semaphore S as a binary semaphore

- Σ Provides mutual exclusion

Semaphore mutex; // initialized to 1

do {

 wait (mutex);

 // Critical Section

 signal (mutex);

 // remainder section

} while (TRUE);

Semaphore Implementation

- Σ Must guarantee that no two processes can execute wait () and signal () on the same semaphore at the same time
- Σ Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
 - 4 But implementation code is short
 - 4 Little busy waiting if critical section rarely occupied
 - 4 Not that applications may spend lot of time in critical sections and therefore this is not a good solution

Deadlock and Starvation

- Σ **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Σ Let S and Q be two semaphores initialized to 1

$P_0 P_1$

wait(S);

wait (Q);

wait(Q);

wait (S);

. .

signal(S);

signal (Q);

signal(Q);

signal (S);

Σ **Starvation** – indefinite blocking

- A process may never be removed from the semaphore queue in which it is suspended

Σ **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process

- Solved via **priority-inheritance protocol**

Classical Problems of Synchronization:

Σ Classical problems used to test newly-proposed synchronization schemes

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem

Bounded-Buffer Problem

Σ N buffers, each can hold one item

Σ Semaphore mutex initialized to the value 1

Σ Semaphore full initialized to the value 0

Σ Semaphore empty initialized to the value N

Σ The structure of the producer process

do {

 // produce an item in

 nextpwait (empty);

 wait(mutex);

 // add the item to the buffer

 signal (mutex);

 signal (full);

 } while (TRUE);

```

    Σ The structure of the consumer process
    do {
wait (full);
wait (mutex);

        // remove an item from buffer to nextc

signal (mutex);
signal (empty);

        // consume the item in nextc

    } while (TRUE);

```

Readers-Writers Problem:

```

    Σ A data set is shared among a number of concurrent processes
        ○ Readers – only read the data set; they do not perform any updates
        ○ Writers – can both read and write

    Σ Problem – allow multiple readers to read at the same time
        ○ Only one single writer can access the shared data at the same time
        ○ Several variations of how readers and writers are treated – all involve priorities

    Σ Shared Data
        ○ Data set
        ○ Semaphore mutex initialized to 1
        ○ Semaphore wrt initialized to 1
        ○ Integer readcount initialized to 0

    Σ The structure of a writer process
do {
wait (wrt) ;

        // writing is performed

signal (wrt) ;

    } while (TRUE);

    Σ The structure of a reader process

```

```

wait (mutex)
;readcount ++
;
if (readcount == 1)    wait (wrt) ;

signal (mutex)
    // reading is performed
wait (mutex) ;
readcount - -;
if (readcount == 0)
    signal (wrt) ;
signal (mutex) ;
    } while (TRUE);

```

Dining-Philosophers Problem



- Σ Philosophers spend their lives thinking and eating
- Σ Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- Σ In the case of 5 philosophers
 - Shared data

4Bowlofrice(dataset)

Semaphorechopstick[5]initializedto1

Σ The structure of Philosophi:

```

wait ( chopstick[i] );

    wait ( chopStick[ (i + 1) % 5] );

    // eat

    signal ( chopstick[i] );

    signal ( chopstick[ (i + 1) % 5] );

// think
} while (TRUE);

```

Monitors

- Σ A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Σ *Abstract data type*, internal variables only accessible by code within the procedure
- Σ Only one process may be active within the monitor at a time
- Σ But not powerful enough to model some synchronization schemes

monitor monitor-name

```

{

    // shared variable declarations

    procedure P1 (...) { .... }

    procedure Pn (...) { ..... }

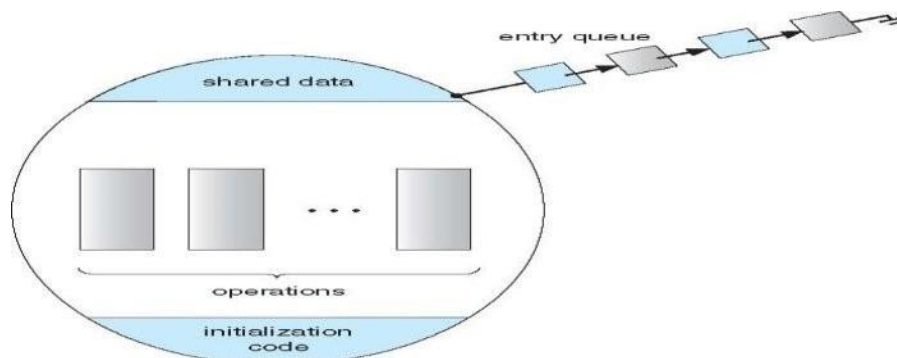
    Initialization code (...) { ... }

}

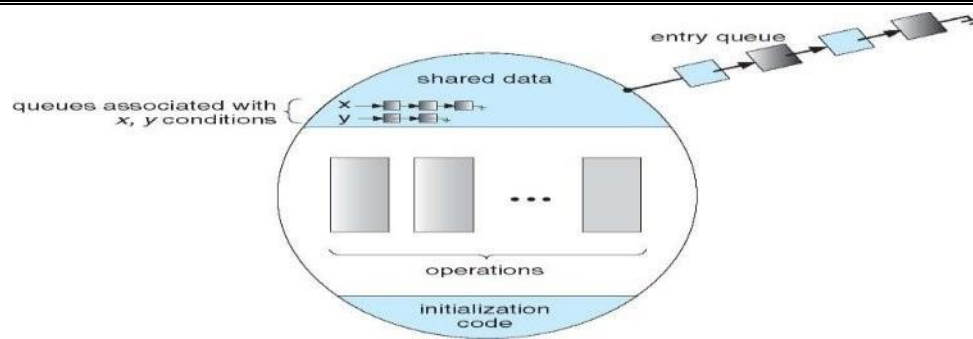
}

```

Schematic view of a Monitor



Monitor with Condition Variables



Scheduling Criteria:

- Σ **CPU utilization** – keep the CPU as busy as possible
- Σ **Throughput** – # of processes that complete their execution per time unit
- Σ **Turnaround time** – amount of time to execute a particular process
- Σ **Waiting time** – amount of time a process has been waiting in the ready queue
- Σ **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithm Optimization Criteria

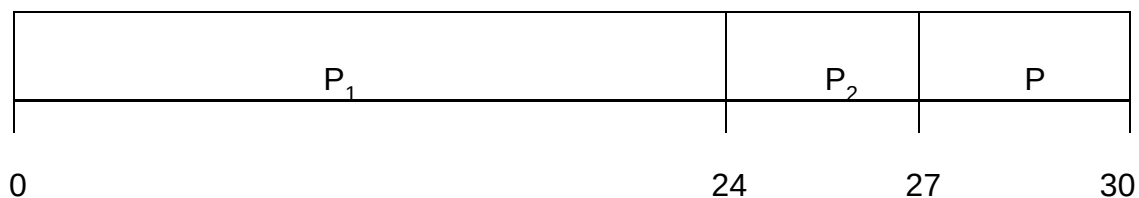
- Σ Max CPU utilization
- Σ Max throughput
- Σ Min turnaround time
- Σ Min waiting time
- Σ Min response time

First-Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

Suppose that the processes arrive in the order: P_1, P_2, P_3

The Gantt Chart for the schedule is:



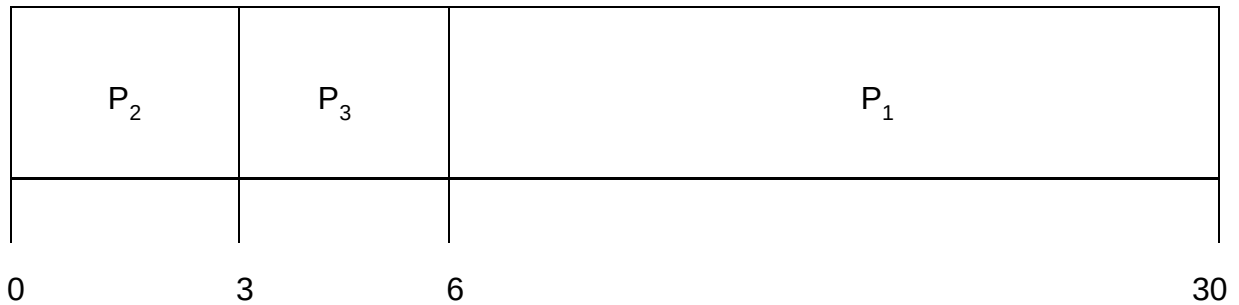
Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$

Σ Average waiting time: $(0 + 24 + 27)/3 = 17$

Σ Suppose that the processes arrive in the order:

P_2, P_3, P_1

Σ The Gantt chart for the schedule is:



Σ Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$

Σ Average waiting time: $(6 + 0 + 3)/3 = 3$

Σ Much better than previous case

Σ **Convoy effect** - short process behind long process

- Consider one CPU-bound and many I/O-bound processes

Shortest-Job-First (SJF) Scheduling

Σ Associate with each process the length of its next CPU burst

- Use these lengths to schedule the process with the shortest time

Σ SJF is optimal – gives minimum average waiting time for a given set of processes

- The difficulty is knowing the length of the next CPU request
- Could ask the user Process Arrival

Time Burst Time

P ₁	0.0	6
P ₂	2.0	8
P ₃	4.0	7
P ₄	5.0	3

SJF scheduling chart

$$\Sigma \text{ Average waiting time} = (3 + 16 + 9 + 0) / 4 = 7$$

Priority Scheduling

- Σ A priority number (integer) is associated with each process
- Σ The CPU is allocated to the process with the highest priority (smallest integer) highest priority)
 - Preemptive
 - Nonpreemptive
 - SJF is priority scheduling where priority is the inverse of predicted next CPU burst time

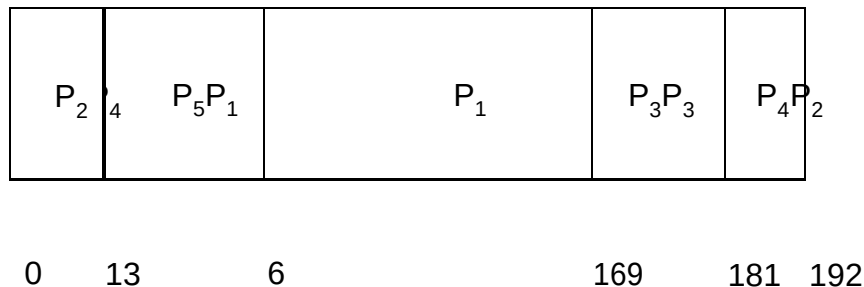
Σ Problem **Starvation**– low priority processes may never execute

Σ Solution **Aging**– as time progresses increase the priority of the

process

P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Priority scheduling Gantt Chart



Average waiting time = 8.2 msec